

Ingegneria del Software

Prova Scritta del 11 Luglio 2025 – bozza di soluzione

Tempo a disposizione: 30 minuti

Esercizio Gruppo 1

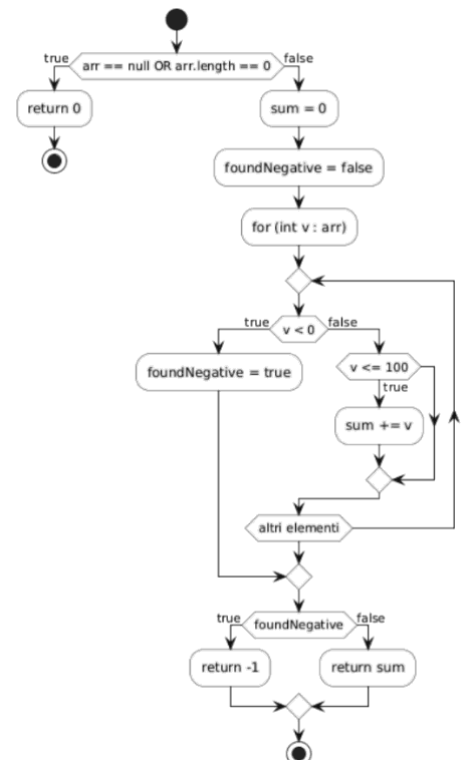
Dato il seguente codice Java:

```
public class AnalisiArray {  
    /**  
     * Restituisce:  
     * - 0 se l'array è nullo o vuoto  
     * - -1 se contiene valori negativi  
     * - la somma dei valori <= 100 altrimenti  
     * Ignora valori > 100.  
     */  
    public int analizza(int[] arr) {  
        if (arr == null || arr.length == 0) {  
            return 0;  
        }  
        int sum = 0;  
        boolean foundNegative = false;  
  
        for (int v : arr) {  
            if (v < 0) {  
                foundNegative = true;  
                break;  
            }  
            if (v <= 100) {  
                sum += v;  
            }  
        }  
        if (foundNegative) {  
            return -1;  
        } else {  
            return sum;  
        }  
    }  
}
```

1. **Disegna il Control Flow Graph** del metodo **analizza()**, identificando nodi e rami
2. **Scrivi una suite di test JUnit 5** che copra ogni possibile ramo (ovvero che soddisfi il criterio di branch coverage)
3. **Discutere l'eventuale minimalità della testsuite proposta.**
Una test suite minimale è un insieme essenziale e non ridondante di casi di test che: Copre un obiettivo specifico di testing (es. *branch coverage* o *statement coverage*), È composta da il minor numero possibile di test, Evita test duplicati o superflui (che non aggiungono copertura)

Rami principali

- `arr == null`
- `arr.length == 0`
- all'interno del ciclo:
 - `v < 0` (porta al break e al ramo `foundNegative`)
 - `v <= 100` (aggiunge al sum)
 - `v > 100` (case ignorato, continua il ciclo)
- Dopo il ciclo:
 - `foundNegative == true` → `return -1`
 - `foundNegative == false` → `return sum`



Casi di test JUnit 5 (non minimale)

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class AnalisiArrayTest {
    private final AnalisiArray a = new AnalisiArray();
    // per la brach covearage non sono necessari entrambi i primi due casi di test (ne basta uno)
    @Test
    public void testNullArray() {
        assertEquals(0, a.analizza(null));
    }
    @Test
    public void testEmptyArray() {
        assertEquals(0, a.analizza(new int[] {}));
    }
    @Test
    public void testNegativeAtStart() {
        assertEquals(-1, a.analizza(new int[] { -5, 10, 20 }));
    }
    @Test
    public void testNegativeMidLoop() {
        assertEquals(-1, a.analizza(new int[] { 10, -3, 20 }));
    }
    @Test
    public void testAllPositiveUnder100() {
        assertEquals(15, a.analizza(new int[] { 5, 10 }));
    }
    @Test
    public void testMixedWithAbove100() {
        assertEquals(15, a.analizza(new int[] { 110, 5, 10, 200 }));
    }
}
```

Casi di test JUnit 5 (minimale)

```
import static org.junit.Assert.*;
import org.junit.Test;

public class AnalisiArrayTest {

    AnalisiArray a = new AnalisiArray();

    @Test
    public void testNull() {
        assertEquals(0, a.analizza(null));
    }

    @Test
    public void testValoriValidiEMaggioriDi100() {
        assertEquals(90, a.analizza(new int[] { 90, 101 }));
    }

    @Test
    public void testValoreNegativo() {
        assertEquals(-1, a.analizza(new int[] { -3 }));
    }
}
```

Discussione

I test coprono:

Ramo	Coperto da
<code>arr == null</code>	<code>null</code>
<code>arr.length == 0</code>	—
<code>v < 0 → true</code>	<code>{-3}</code>
<code>v < 0 → false</code>	<code>{90, 101}</code>
<code>v <= 100 → true</code>	<code>{90, 101}</code>
<code>v <= 100 → false</code>	<code>{90, 101}</code>
<code>foundNegative == true</code>	<code>{-3}</code>
<code>foundNegative == false</code>	<code>{90, 101}</code>

L'unico **ramo non coperto** è `arr.length == 0`, ma **non serve ai fini della branch coverage**, perché è **fuso con `arr == null`** nel costrutto:

```
if (arr == null || arr.length == 0)
```

Questo è **una sola decisione (OR)**, e nel test `null` il ramo `arr == null` è già coperto come `true`, quindi l'intero `if` va a `true → coperto`.